



Departamento de Gestión de Procesos y Sistemas.

Facultad de Ingeniería.

Materia: Base de Datos I

Trimestre: 202324_1

Secciones: FPTSP01-01 & FPTSP01-02

Profesora: Concettina Di Vasta Valente

Proyecto - Fase 2

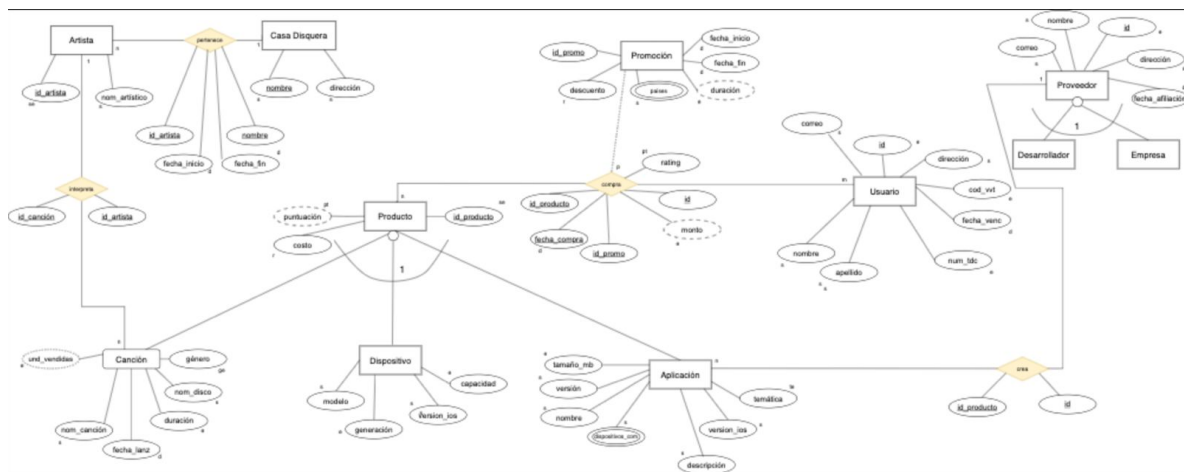
Integrantes:

Victor Pointud

Carnet: 20221110061

CI: 30.124.022

Diagrama E.R



Transformación de Modelo Entidad Relación a Modelo Relacional

Cumpliendo La Primera Forma Normal (1FN):

1. **Artista** (nombre, id_artista, nom_artístico , fecha_inicio, fecha_fin).
2. **Casa disquera**(nombre, ciudad, municipio, calle, cod_postal, cod_viv).
 - a) Cómo dirección es un atributo compuesto se separó y se volvió atómico con ciudad, municipio, calle, cod_postal y cod_viv.
3. **Producto**(id_producto, costo, puntuación).
4. **Canción**(id_producto, id_cancion, id_artista, costo, und_vendidas [atr derivado], nomb_canción, nomb_disco, fecha_lanz, duración, género, puntuación [atr derivado]).
5. **Dispositivo**(id_producto, costo, modelo, generación, versión_ios, capacidad, puntuación [atr derivado]).
6. **Aplicación**(id_producto, id, costo, tamaño_mb, versión, nombre, descripción, temática, versión_ios, puntuación [atr derivado]).
7. **Dispositivos compatibles**(id_producto, dispositivo) [atr multivaluado de aplicación].
8. **Proveedor**(id, nombre, apellido, correo, fecha_afil, ciudad, municipio, calle, cod_postal, cod_viv, tipo_proveedor [empresa o desarrollador]).
 - a) Cómo dirección es un atributo compuesto se separó y se volvió atómico con ciudad, municipio, calle, cod_postal y cod_viv.
 - b) Cómo nombre es un atributo compuesto se separó y se volvió atómico con nombre y apellido.
9. **Usuario**(id, país, ciudad, municipio, calle, cod_postal, cod_viv, fecha_venc, num_trj, nombre, apellido, correo).
 - a) Cómo dirección es un atributo compuesto se separó y se volvió atómico con ciudad, municipio, calle, cod_postal y cod_viv.
10. **Promoción**(id_promo, descuento, fecha_inicio, fecha_fin, duración [atr derivado]).
11. **Países**(id_promo, país) [atr multivaluado de promoción].
12. **Compra**(id, id_producto, id_promo, fecha_compra, rating, monto [atr derivado]).

Cumpliendo La Segunda Forma Normal (2FN):

1. **Artista** (nombre, id_artista, nom_artistico , fecha_inicio, fecha_fin).
2. **Casa disquera**(nombre, ciudad, municipio, calle, cod_postal, cod_viv).
 - a) Ciudad se convierte en determinante dado que municipio, calle, cod_postal y cod_viv dependen de ella.
3. **Producto**(id_producto, costo, puntuación).
4. **Canción**(id_producto, id_cancion, id_artista, costo, und_vendidas [atr derivado], nomb_cancion, nomb_disco, fecha_lanz, duración, género, puntuación [atr derivado]).
5. **Dispositivo**(id_producto, costo, modelo, generación, versión_ios, capacidad, puntuación [atr derivado]).
6. **Aplicación**(id_producto, id, costo, tamaño_mb, versión, nombre, descripcion, temática, version_ios, puntuación [atr derivado]).
7. **Dispositivos compatibles**(id_producto, dispositivo) [atr multivaluado de aplicación].
8. **Proveedor**(id, nombre, apellido, correo, fecha_afil, ciudad, municipio, calle, cod_postal, cod_viv, tipo_proveedor [empresa o desarrollador]).
 - a) Ciudad se convierte en determinante dado que municipio, calle, cod_postal y cod_viv dependen de ella.
9. **Usuario**(id, país, ciudad, municipio, calle, cod_postal, cod_viv, fecha_venc, num_trj, nombre, apellido, correo).
 - a) Ciudad se convierte en determinante dado que municipio, calle, cod_postal y cod_viv dependen de ella.
10. **Promoción**(id_promo, descuento, fecha_inicio, fecha_fin, duración [atr derivado]).
11. **Países**(id_promo, país) [atr multivaluado de promoción].
12. **Compra**(id, id_producto, id_promo, fecha_compra, rating, monto [atr derivado]).

Cumpliendo La Tercera Forma Normal (3FN):

1. **Artista** (nombre, id_artista, nom_artístico , fecha_inicio, fecha_fin).
2. **Casa disquera**(nombre, id_ciudad_casa_disq).
3. **Producto**(id_producto, costo, puntuación).
4. **Canción**(id_producto, id_canción, id_artista, costo, und_vendidas [atr derivado], nomb_cancion, nomb_disco, fecha_lanz, duración, género, puntuación [atr derivado]).
5. **Dispositivo**(id_producto, costo, modelo, generación, versión_ios, capacidad, puntuación [atr derivado]).
6. **Aplicación**(id_producto, id, costo, tamaño_mb, versión, nombre, descripción, temática, versión_ios, puntuación [atr derivado]).
7. **Dispositivos compatibles**(id_producto, dispositivo) [Esta relación proviene de un atributo multivaluado de dispositivos, que al pasar al modelo relacional se crea su propia tabla.]
8. **Proveedor**(id_prov, id_ciudad_prov, nombre, apellido, correo, fecha_afil, tipo_proveedor [empresa o desarrollador]).
9. **Usuario**(id, país, id_ciudad_usu, fecha_venc, num_trj, cod_vvt,nombre, apellido, correo).
10. **Promoción**(id_promo, descuento, fecha_inicio, fecha_fin, duración [atr derivado]).
11. **Países**(id_promo, país) [atr multivaluado de promoción].
12. **Compra**(id, id_producto, id_promo, fecha_compra, rating, monto [atr derivado]).
13. **Ciudad casa disquera**(id_ciudad_casa_disq, nombre, municipio, calle, cod_postal, cod_viv).
 - a) Se crea tabla Ciudad Disquera ya hay una dependencia transitiva en Casa Disquera que contiene id_ciudad_disq como propio PK junto a nombre es PK de Casa Disquera, finalizando con sus atributos.
14. **Ciudad proveedor**(id_ciudad_prov, id_prov, municipio, calle, cod_postal, cod_viv).
 - a) Se crea tabla Ciudad Proveedor ya hay una dependencia transitiva en Proveedor que contiene id_ciudad_prov como propio PK junto a id que es PK de Proveedor respectivamente. Finalizando con sus atributos.

15. **Ciudad usuario**(id_ciudad_usu, id, municipio, calle, cod_postal, cod_viv).

- a) Se crea tabla Ciudad Usuario ya hay una dependencia transitiva en Usuario que contiene id_ciudad_usu como propio PK junto a id que es PK de Usuario respectivamente. Finalizando con sus atributos.

Inserciones Solicitadas

Creación de tablas:

1. Artista

```
CREATE TABLE "ARTISTA"(  
  id_artista SERIAL NOT NULL,  
  nombre VARCHAR(100) NOT NULL,  
  nom_artístico VARCHAR(100) NOT NULL,  
  fecha_inicio DATE NOT NULL,  
  fecha_fin DATE,  
  CONSTRAINT artista_pk PRIMARY KEY (id_artista)  
  CONSTRAINT casa_disquera_fk FOREIGN KEY(nombre) REFERENCES "CASA  
DISQUERA"(nombre)  
);
```

2. Casa disquera

```
CREATE TABLE "CASA DISQUERA"(  
  nombre VARCHAR(100) NOT NULL,  
  id_ciudad_casa_disq SERIAL NOT NULL,  
  CONSTRAINT casa_disquera_pk PRIMARY KEY(nombre),  
  CONSTRAINT ciudad_casa_disq_fk FOREIGN KEY (id_ciudad_casa_disq),  
  REFERENCES "CIUDAD CASA DISQUERA"(id_ciudad_casa_disq),  
);
```

3. Ciudad de casa disquera

```
CREATE TABLE "CIUDAD CASA DISQUERA" (
```

```

id_ciudad_casa_disq SERIAL NOT NULL,
nombre VARCHAR(100) NOT NULL,
municipio VARCHAR(100) NOT NULL,
calle VARCHAR(100) NOT NULL,
cod_postal VARCHAR(10) NOT NULL,
cod_viv VARCHAR(10) NOT NULL,
CONSTRAINT ciudad_casa_disquera_pk PRIMARY KEY (id_ciudad_casa_disq ),
CONSTRAINT casa_disquera_fk FOREIGN KEY (nombre) REFERENCES "CASA
DISQUERA"(nombre)
);

```

4. Producto

```

CREATE TABLE "PRODUCTO"(
id_producto SERIAL NOT NULL, is
costo INT NOT NULL,
puntuación INT NOT NULL,
CONSTRAINT producto_pk PRIMARY KEY (id_producto)
);

```

5. Usuario

```

CREATE TABLE "USUARIO"(
id SERIAL NOT NULL,
país VARCHAR(100) NOT NULL,
id_ciudad_usu SERIAL NOT NULL,
fecha_venc DATE NOT NULL,
num_tdc VARCHAR(15) NOT NULL,
cod_vvt INT NOT NULL,
nombre VARCHAR(100) NOT NULL,
apellido VARCHAR(100) NOT NULL,
correo VARCHAR(100) NOT NULL,
CONSTRAINT usuario_pk PRIMARY KEY (id),
CONSTRAINT ciudad_usuario_fk FOREIGN KEY (id_ciudad_usu) REFERENCES
"CIUDAD USUARIO"(id_ciudad_usu),
CONSTRAINT países_fk FOREIGN KEY (país) REFERENCES "PAÍSES"(país)
);

```

6. Ciudad usuario

```

CREATE TABLE "CIUDAD_USUARIO"(

```

```

id_ciudad_usu SERIAL NOT NULL,
id SERIAL NOT NULL,
municipio VARCHAR(100) NOT NULL,
calle VARCHAR(100) NOT NULL,
cod_postal VARCHAR(10) NOT NULL,
cod_viv VARCHAR(10) NOT NULL,
CONSTRAINT ciudad_usuario_pk PRIMARY KEY (id_ciudad_usu),
CONSTRAINT usuario_fk FOREIGN KEY (id) REFERENCES "USUARIO"(id)
);

```

7. Promoción

```

CREATE TABLE "PROMOCIÓN"(
id_promo SERIAL NOT NULL,
descuento REAL NOT NULL,
fecha_inicio DATE NOT NULL,
fecha_fin DATE NOT NULL,
duración INT GENERATED ALWAYS AS (EXTRACT(EPOCH FROM (fecha_fin -
fecha_inicio)) / 86400) STORED,
CONSTRAINT promoción_pk PRIMARY KEY (id_promo)
);

```

8. Compra

```

CREATE TABLE "COMPRA"(
fecha_compra DATE NOT NULL,
id_producto SERIAL NOT NULL,
id_promo SERIAL NOT NULL,
id SERIAL NOT NULL,
rating INT NOT NULL,
monto INT GENERATED ALWAYS AS ( SELECT costo FROM Producto WHERE
id_producto = Compra.id_producto) STORED,
CONSTRAINT compra_pk PRIMARY KEY (fecha_compra),
CONSTRAINT usuario_fk FOREIGN KEY (id) REFERENCES "USUARIO"(id),
CONSTRAINT promoción_fk FOREIGN KEY (id_promo) REFERENCES
"PROMOCIÓN"(id),
CONSTRAINT producto_fk FOREIGN KEY (id_producto) REFERENCES
"PRODUCTO"(id_producto)
);

```

```

CREATE OR REPLACE FUNCTION calcular_descuento_function() RETURNS trigger AS
$$
BEGIN
    IF NEW.id_promo IS NOT NULL

```

```

        THEN
            NEW.monto = (SELECT costo FROM "PRODUCTO" WHERE id_producto =
NEW.id_producto) - ((SELECT costo FROM "PRODUCTO" WHERE id_producto =
NEW.id_producto) * (SELECT descuento FROM "PROMOCIÓN" WHERE id_promo =
NEW.id_promo) / 100);
        ELSE
            NEW.monto = (SELECT costo FROM "PRODUCTO" WHERE id_producto =
NEW.id_producto);
        END IF;
        RETURN NEW;
    END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER calcular_descuento_trigger
BEFORE INSERT ON "COMPRA"
FOR EACH ROW
EXECUTE FUNCTION calcular_descuento_function();

```

9. Proveedor

```

CREATE TABLE "PROVEEDOR"(
    id SERIAL NOT NULL,
    id_ciudad_prov SERIAL NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    correo VARCHAR(100) NOT NULL,
    fecha_afil DATE NOT NULL,
    tipo_proveedor VARCHAR(20) CHECK (tipo_proveedor IN ('empresa', 'desarrollador')),
    CONSTRAINT ciudad_proveedor_fk FOREIGN KEY (id_ciudad_prov) REFERENCES
"CIUDAD PROVEEDOR"(id_ciudad_prov)
);

```

10. Ciudad proveedor

```

CREATE TABLE "CIUDAD_PROVEEDOR" (

```

```

id_ciudad_prov SERIAL NOT NULL,
id SERIAL NOT NULL,
municipio VARCHAR(255),
calle VARCHAR(255),
cod_postal VARCHAR(10),
cod_viv VARCHAR(10),
CONSTRAINT ciudad_prov_pk PRIMARY KEY (id_ciudad_prov),
CONSTRAINT proveedor_fk FOREIGN KEY (id) REFERENCES "PROVEEDOR"(id)
);

```

11. Países

```

CREATE TABLE "PAÍSES" (
    país VARCHAR(20) NOT NULL,
    id_promo SERIAL NOT NULL,
    CONSTRAINT promoción_fk FOREIGN KEY (id_promo) REFERENCES
"Promoción"(id_promo)
);

```

12. Aplicación

```

CREATE TABLE "APLICACIÓN" (
    id_producto SERIAL NOT NULL,
    id SERIAL NOT NULL,
    costo REAL NOT NULL,
    tamaño_mb INT NOT NULL,
    versión REAL NOT NULL,
    nombre VARCHAR NOT NULL,
    descripción VARCHAR NOT NULL,
    temática VARCHAR NOT NULL,
    versión_ios REAL NOT NULL,
    puntuación INT,
    CONSTRAINT aplicación_pk PRIMARY KEY (id),
    CONSTRAINT producto_fk FOREIGN KEY (id_producto) REFERENCES
"PRODUCTO"(id_producto)
);

```

13. Canción

```

CREATE TABLE "CANCIÓN" (

```

```

id_producto SERIAL NOT NULL,
id_canción SERIAL NOT NULL,
id_artista SERIAL NOT NULL,
costo REAL NOT NULL,
und_vendidas INT GENERATED ALWAYS AS (/ * fórmula para calcular las unidades
vendidas */),
nomb_canción VARCHAR NOT NULL,
nomb_disco VARCHAR NOT NULL,
fecha_lanz DATE NOT NULL,
duración INT NOT NULL,
género VARCHAR NOT NULL,
puntuación INT,
CONSTRAINT canción_pk PRIMARY KEY (id_canción),
CONSTRAINT producto_fk FOREIGN KEY (id_producto) REFERENCES
"PRODUCTO"(id_producto),
CONSTRAINT artista_fk FOREIGN KEY (id_artista) REFERENCES
"ARTISTA"(id_artista)
);

```

14. Dispositivo

```

CREATE TABLE "DISPOSITIVO" (
id_producto SERIAL NOT NULL,
costo REAL NOT NULL,
modelo VARCHAR NOT NULL,
generación VARCHAR NOT NULL,
versión_ios REAL NOT NULL,
capacidad INT NOT NULL,
puntuación INT,
CONSTRAINT dispositivo_pk PRIMARY KEY (id_producto),
CONSTRAINT producto_fk FOREIGN KEY (id_producto) REFERENCES
"PRODUCTO"(id_producto)
);

```

15. Dispositivos_Comp

```

CREATE TABLE "DISPOSITIVOS_COMP" (
id_producto SERIAL NOT NULL,
dispositivo VARCHAR(100) NOT NULL,
CONSTRAINT producto_fk FOREIGN KEY (id_producto) REFERENCES
"PRODUCTO"(id_producto)
);

```

Inserción de datos:

Nosotros como equipo usamos la app Dbeaver como SMBD donde posee una gran habilidad integrada para importar datos tipo CVS en la base de datos, en nuestro caso usamos google sheets para rellenar los atributos de las tablas con la información pertinente y luego la introducimos en la base de datos, a continuación le dejamos el link para su visualización:

<https://docs.google.com/spreadsheets/d/1KjSuHAb4I72vHuLvIkpTiMiuLFziZRlgQxRQ1sl9b0k/edit?usp=sharing>

Igualmente aquí hay un extracto de código de como Dbeaver se encarga de pasar los datos:

```
COPY proyecto."USUARIO" (id, fecha_venc, nombre, apellido, correo,
id_ciudad_usu, num_tdc, cod_vvt) FROM stdin;
1      2026-06-05  María López maria@gmail.com  1      123456789012
342
2      2024-03-12  Juan  Martínez   juan@hotmail.com  2      987654321098
587
3      2026-10-03  Ana   García      ana@gmail.com     3      567890123456
291
4      2025-06-08  CarlosPérez carlos@hotmail.com  4      345678901234
653
5      2023-10-09  Diego Rodríguez  diego@gmail.com   5      901234567890
800
6      2025-05-11  Laura Sánchez   laura@hotmail.com 6      678901234567
364
7      2025-05-04  Martín Díaz    martin@gmail.com  7      432109870543
211
8      2025-04-07  Sofía Fernández sofia@hotmail.com  8      210987654321
134
9      2025-12-01  Pablo Torrespablo@gmail.com  9      789012345678
590
10     2023-08-06  Lucia Ruiz     lucia@hotmail.com 10     654321098765
837
\.
```

Restricciones:

Los precios de los productos deben ser mayores a 0, así como no pueden existir descuentos negativos ni mayores al 100%.

- 1. La fecha de finalización de una promoción no puede ser menor a la fecha de inicio de tal promoción:**

```
ALTER TABLE "PROMOCIÓN"  
ADD CONSTRAINT fechas_check  
CHECK (fecha_fin >= fecha_inicio);
```

- 2. La duración de las canciones no puede ser menor a cero, de igual forma el tamaño (MB) de las aplicaciones.**

```
ALTER TABLE "CANCIÓN"  
ADD CONSTRAINT duración_check  
CHECK (duración >= 0);
```

```
ALTER TABLE "APLICACIÓN"  
ADD CONSTRAINT tamaño_mb_check  
CHECK (tamaño_mb >= 0);
```

- 3. Si un usuario ha comprado más de 3 productos debe crearse automáticamente una promoción que le ofrezca un descuento en su compra del 30%, que comience el mismo día de la compra y termine pasado un mes, con el fin de incentivar su consumo.**

```
CREATE FUNCTION crear_promoción_si_necesario_function()  
RETURNS TRIGGER AS $$  
DECLARE cantidad_compras INTEGER;  
BEGIN  
    SELECT COUNT(*) INTO cantidad_compras  
    FROM "COMPRA"  
    WHERE id = NEW.id;  
    IF cantidad_compras > 3 THEN  
        INSERT INTO "PROMOCIÓN" (descuento, fecha_inicio, fecha_fin)  
        VALUES (0.3, CURRENT_DATE, CURRENT_DATE + INTERVAL '1 month');  
    END IF;  
  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER crear_promoción_si_necesario_trigger
AFTER INSERT ON "COMPRA"
FOR EACH ROW
EXECUTE FUNCTION crear_promoción_si_necesario_function();
```

4. **Se necesita que a la hora de comprar una aplicación, la versión del iOS del Dispositivo que posee el cliente sea mayor o igual a la versión del iOS que especifica dicha aplicación, en caso de no ser así no podrá efectuarse la compra.**

```
CREATE OR REPLACE FUNCTION versión_ios_check_function()
RETURNS TRIGGER AS $$
DECLARE max_app_versión FLOAT;
BEGIN
    SELECT MAX(versión_ios) INTO max_app_versión
    FROM "APLICACIÓN"
    WHERE id_producto = NEW.id_producto;
    IF max_app_versión IS NOT NULL AND (
        SELECT versión_ios
        FROM "DISPOSITIVO"
        WHERE id_producto = NEW.id_producto) >= max_app_versión
    THEN
        RETURN NEW;
    ELSE
        RAISE EXCEPTION 'La versión iOS del dispositivo no es mayor o igual a
todas las versiones de la aplicación asociadas al producto';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER versión_ios_check_trigger
BEFORE INSERT OR UPDATE ON "COMPRA"
FOR EACH ROW
EXECUTE FUNCTION versión_ios_check_function();
```

5. **Las puntuaciones que dan los usuarios a los productos comprados deben estar entre 0 y 5 puntos (enteros).**

```
ALTER TABLE proyecto."CANCIÓN" (
```

```

ADD CONSTRAINT puntuación_check CHECK (puntuación >= 0 AND puntuación
<= 5);
);
ALTER TABLE proyecto."DISPOSITIVO"(
ADD CONSTRAINT puntuación_check CHECK (puntuación >= 0 AND puntuación
<= 5);
);
ALTER TABLE proyecto."APLICACIÓN"(
ADD CONSTRAINT puntuación_check CHECK (puntuación >= 0 AND puntuación
<= 5);
);

```

- 6. La puntuación total de un producto debe ser calculada como el promedio de los rating que dejaron los usuarios en sus compras.**

```

CREATE OR REPLACE FUNCTION actualizar_puntuación_function()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE "PRODUCTO"
    SET puntuación = (SELECT AVG(rating)
    FROM "COMPRA"
    WHERE "COMPRA".id_producto = NEW.id_producto)
    WHERE "PRODUCTO".id_producto = NEW.id_producto;
    UPDATE "DISPOSITIVO"
    SET puntuación = (SELECT AVG(rating)
    FROM "COMPRA"
    WHERE "COMPRA".id_producto = NEW.id_producto)
    WHERE "DISPOSITIVO".id_producto = NEW.id_producto;
    UPDATE "APLICACIÓN"
    SET puntuación = (SELECT AVG(rating)
    FROM "COMPRA"
    WHERE "COMPRA".id_producto = NEW.id_producto)
    WHERE "APLICACIÓN".id_producto = NEW.id_producto;
    UPDATE "CANCIÓN"
    SET puntuación = (SELECT AVG(rating)
    FROM "COMPRA"
    WHERE "COMPRA".id_producto = NEW.id_producto)
    WHERE "CANCIÓN".id_producto = NEW.id_producto;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER after_insert_actualizar_puntuación

```

```
AFTER INSERT ON "COMPRA"
FOR EACH ROW
EXECUTE FUNCTION actualizar_puntuación_function();
```

7. **Se debe validar al momento de aplicar una promoción en una compra que el país del usuario coincida con los países de la promoción.**

```
CREATE OR REPLACE FUNCTION valida_país_promoción_function() RETURNS
TRIGGER AS $$
BEGIN
    IF EXISTS (SELECT 1
                FROM "PAÍSES" P
                JOIN "USUARIO" U ON NEW.id = U.id
                WHERE P.id_promo = NEW.id_promo AND P.país = U.país)
    THEN
        RETURN NEW;
    ELSE
        RAISE EXCEPTION 'El país del usuario no está asociado con la
        promoción';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER valida_país_promoción_trigger
BEFORE INSERT OR UPDATE ON "COMPRA"
FOR EACH ROW
EXECUTE FUNCTION valida_país_promoción_function();
```

8. **Tomar en cuenta dominios personalizados como género musical asociado como: Pop, Balada, Rock, entre otras y temática de aplicaciones: juegos, cocina, lectura, entre otros.**

```
ALTER TABLE "CANCIÓN"
ADD CONSTRAINT género_check
CHECK (género IN ('Pop', 'Balada', 'Rock') OR género IS NOT NULL);
```

```
ALTER TABLE "APLICACIÓN"
ADD CONSTRAINT temática_check
CHECK (temática IN ('Juegos', 'Cocina', 'Lectura') OR temática IS NOT NULL);
```

Consultas SQL:

1. **Encontrar el desarrollador o compañía que ha producido más aplicaciones en la tienda.**

```

SELECT tipo_proveedor, p.nombre, COUNT(*) as cantidad_apps
FROM proyecto."PROVEEDOR" p
JOIN proyecto."APLICACIÓN" a ON p.id = a.id
GROUP BY tipo_proveedor, p.nombre
ORDER BY cantidad_apps DESC
LIMIT 1;

```

2. **Listar las tarjetas de crédito que están próximas a vencer en los próximos 3 meses y mostrar cuántas compras se han realizado con ellas.**

```

SELECT u.num_tdc, u.fecha_venc, count(c.id_producto) AS compras_realizadas
FROM proyecto."USUARIO" u
JOIN proyecto."COMPRA" c ON u.id = c.id
WHERE u.id = c.id AND u.fecha_venc BETWEEN current_date AND current_date
+ interval '+3 month'
GROUP BY u.num_tdc, u.fecha_venc;

```

3. **Listar los usuarios que compraron aplicaciones de la temática 'cocina' y que también compraron canciones del género 'electrónica'.**

```

SELECT DISTINCT u.id, u.nombre, u.apellido, ca.género as tipo
FROM proyecto."USUARIO" u
JOIN proyecto."COMPRA" c1 ON u.id = c1.id
JOIN proyecto."CANCIÓN" ca ON c1.id_producto = ca.id_producto
WHERE ca.género = 'Electrónica' OR ca.género = 'electrónica'
UNION
SELECT DISTINCT u.id, u.nombre, u.apellido, a.temática as tipo
FROM proyecto."USUARIO" u
JOIN proyecto."COMPRA" c1 ON u.id = c1.id
JOIN proyecto."APLICACIÓN" a ON c1.id_producto = a.id_producto
where a.temática = 'Cocina' OR a.temática = 'cocina'

```

4. **Listar las aplicaciones que son compatibles con la versión más reciente de iOS y que tienen una puntuación superior a 4.**

```

SELECT *
FROM "APLICACIÓN"
WHERE versión_ios = (SELECT MAX(versión_ios) FROM "APLICACIÓN")
AND puntuación > 4;

```

5. **Mostrar los nombres de las casas disqueras que tienen más de 3 artistas, y mostrar la canción más vendida de cada una de esas disqueras, ordenadas por el número de artistas.**

```
SELECT c.nombre AS casa_disquera, max(ca.und_vendidas) as  
max_und_vendidas  
FROM "CASA DISQUERA" c  
JOIN "ARTISTA" ar ON c.nombre = ar.nombre  
JOIN "CANCIÓN" ca ON ar.id_artista = ca.id_artista  
WHERE ((SELECT COUNT(*)  
        FROM "ARTISTA" ar  
        WHERE c.nombre = ar.nombre) > 3)  
GROUP BY c.nombre  
ORDER BY (SELECT COUNT(*)  
          FROM "ARTISTA" ar  
          WHERE c.nombre = ar.nombre) DESC;
```